MAST30027: Modern Applied Statistics

Week 10 Lab

Recall that $\mathbf{X} = \begin{pmatrix} X_1 \\ X_2 \end{pmatrix} \sim N(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, with $\boldsymbol{\mu} = \begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}$ and $\boldsymbol{\Sigma} = \begin{pmatrix} \sigma_1^2 & \sigma_{12} \\ \sigma_{12} & \sigma_2^2 \end{pmatrix}$ has the joint density

$$f_{\boldsymbol{\mu},\boldsymbol{\Sigma}}(\mathbf{x}) = \frac{1}{2\pi|\boldsymbol{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right),$$

where $\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$.

1. Write an R function `dbinorm(x, mu, Sigma)` that evaluates the bivariate normal density $f_{\boldsymbol{\mu},\boldsymbol{\Sigma}}(\mathbf{x})$.

   You will find the functions `solve`, `det`, `t` useful: given a matrix `A` and column vector `b`, `solve(A, b)` computes $\texttt{A}^{-1}\texttt{b}$, `det(A)` computes the determinant of `A`, and `t(b)` transposes the column vector `b` into a row vector. Matrix multiplication in R is done by `%*%`.

   Check your answer by comparing whether `dbinorm(c(1,1), c(0,0), matrix(c(1,0,0,1),2,2))` and `dnorm(1)^2` are equal.

```
dbinorm <- function(x, mu, Sigma) {
  # your code here
}
```

2. Implement the Metropolis-Hastings algorithm to generate a sample of size $n = 2000$ from the $N\left(\begin{pmatrix} 2 \\ 2 \end{pmatrix}, \begin{pmatrix} 4 & 2 \\ 2 & 4 \end{pmatrix}\right)$ distribution. Use the symmetric random walk proposal distribution $N(\mathbf{x}, \sigma^2 I)$ with $\sigma = 1$.

   Use $\mathbf{X}(0) = \begin{pmatrix} 6 \\ -6 \end{pmatrix}$ as your initial state. Report the proportion of accepted values and visualise the sample after discarding the first 100 iterations as burn-in.

```
set.seed(30027)
mu <- c(2, 2)                          # mean
Sigma <- matrix(c(4, 2, 2, 4), 2, 2) # covariance
n_iter <- 2000                         # chain length
init_val <- c(6, -6)                   # initial value
prop_sd <- 1                           # std dev of proposal dist
chain <- matrix(nrow = n_iter+1, ncol = 2) # initialise chain
chain[1,] <- init_val
n_accept <- 0 # counts number of accepted proposals

for (i in 1:n_iter){
  # generate proposal

  # calculate acceptance probability

  # update chain and number of accepted proposals

}
```

3. Let $\mathbf{X}(n)$ be the n-th sample point. Plot $X_i(n)$ and the cumulative averages $\overline{X}_i(n) = \frac{1}{n}\sum_{j=1}^{n} X_i(j)$, for $i = 1, 2$ (over iteration numbers). The cumulative averages should give a rough idea of how quickly $\{\mathbf{X}(n)\}_{n\geq 1}$ converges to the posterior distribution. (Do not discard the burn-in for this question.)